

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1412 COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 20%

Duration :60 Mins

On Class QUESTIONS: 5 Total Marks:50

Annexure A

CLASS TEST

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

CO4-AT1 - CLASS TEST

1. Translation Scheme for Assignment Statement

$$x = (a + b) * (c + d)$$

Three-Address Code (TAC) translation breaks an expression into simple operations using temporary variables.
Translation Scheme:

$$S \rightarrow id = E$$

The general translation is:

$$S.code = E.code \parallel gen(id = E, place)$$

For arithmetic expressions:

$$E \rightarrow E + E2$$

$$E.place = newtemp()$$

$$gen(E.place = E1.place + E2.place)$$

TAC for $x = (a + b) * (c + d)$

$$t1 = a + b$$

$$t2 = c + d$$

$$t3 = t1 * t2$$

$$x = t3$$

∴ Thus, the expression is correctly translated into 4 three-address code.

2. Boolean Expressions & Short-Circuit Evaluation:

Given:

$$(a > b) \&\& (c < d) \parallel (e < f)$$

The grammar includes $\parallel$, $\&\&$, $!$, relational expressions, true & false.

i) Precedence & Associativity:

- $&&$ have higher precedence than $||$.
- $((a > b) \&\& (c < d)) || (e < f)$
- Both $\&\&$ and $||$ are evaluated left to right when they have the same precedence level.

ii) TAC using Short - Circuit Evaluation:

if $a > b$ goto L1

goto L3

L1: if $c < d$ goto L2

goto L3

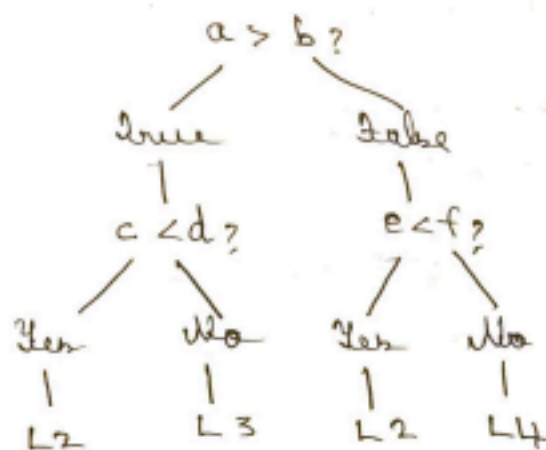
L3: if $e < f$ goto L2

goto L4

L2: /* TRUE */

L4: /* FALSE */

iii) Control Flow:



L2 represents TRUE & L4 represents FALSE.

3. Pointer Operations & Intermediate Code:

Given:

$$x = *y$$

$$a = \&x$$

i) Three-Address Code:

$$t_1 = *y$$

$$x = t_1$$

$$t_2 = \&x$$

$$a = t_2$$

ii) Intermediate Code Representations:

a) Quadruples:

No.	Operator	Argument 1	Argument 2	Result
1	*	y	-	t ₁
2	=	t ₁	-	x
3	&	x	-	t ₂
4	=	t ₂	-	a

b) Triples:

Index	Operator	Argument 1	Argument 2
0	*	y	-
1	=	(0)	x
2	&	x	-
3	=	(2)	a

c) Indirect Triples:

Pointer	Triple Index	Operator	Argument 1	Argument 2
0	0	*	y	-
1	1	=	(0)	x
2	2	&	x	-
3	3	=	(2)	a

4. Three Address Code:

The given program contains an outer while loop, an if-else statement & an inner while

i) Conditional & Unconditional Jump Statements:

if $A < C$ goto L2

goto L8

if $B > D$ goto L3

goto L8

if $A == 1$ goto L4

goto L5

ii) Labels for Given Flow:

L1: $\rightarrow$ Enter while condition

L2: $\rightarrow$ Check $B > D$

L3: $\rightarrow$ Check $A == 1$

L4: $\rightarrow$ Then block

L5: $\rightarrow$ Inner while condition

L6: $\rightarrow$ Inner while body

L8 $\rightarrow$ Exit

iii) Evaluation of Boolean Expressions:

for

while $(A < C \ \&\& \ B > D)$

L1: if $A < C$ goto L2
goto L8

L2: if $B > D$ goto L3
goto L8

for

if $(A == 1)$

L3: if $A == 1$ goto L4
goto L5.

Complete TAC:

L1: if $A < C$ goto L2

goto L8

L2: if $B > D$ goto L3

goto L8

L3: if $A == 1$ goto L4

goto L5

L4: $C = C + 1$

goto L1

L5: if $A <= D$ goto L6

goto L1

L6: $A = A + B$

goto L5

L8: ...

5. Backpatching for Boolean Expression:

Given:

$(a < b) \parallel (c < d) \&\& (e < f)$

Since $\&\&$ has higher precedence than $\parallel$,

$(a < b) \parallel ((c < d) \&\& (e < f))$

i) SDD / Translation Scheme:

Production	Semantic Actions
$B \rightarrow B1$	
$B \rightarrow B1 \&\& B2$	backpatch ($B1$, truelist, $B2$, ibegin)
$B \rightarrow !B1$	B .truelist = $B1$.falselist; B .falselist = $B1$.truelist
$B \rightarrow E1 \vee \text{or } E2$	Generate conditional & unconditional jump create truelist & falselist.
$B \rightarrow \text{true}$	B .truelist = makelist (nextquad)
$B \rightarrow \text{false}$	B .falselist = makelist (nextquad)

ii) TAC Using Short-Circuit Evaluation:

L1: if $a < b$ goto TRUE
goto L2

L2: if $c < d$ goto L3
goto FALSE

L3: if $e < f$ goto TRUE
goto FALSE

TRUE: /* Boolean result = true */

FALSE: /* Boolean result = false */

iii) Truelist, Falselist & Backpatching:

for $a < b$:

Truelist $\rightarrow$ TRUE

Falselist $\rightarrow$ L2

for $c < d$:

Truelist $\rightarrow$ L3

Falselist $\rightarrow$ FALSE

for $e < f$:

Truelist $\rightarrow$ for TRUE

Falselist $\rightarrow$ FALSE

Final Backpatched Code:

Label	Three-Address Code
L1	if $a < b$ goto TRUE goto L2.
L2	if $c < d$ goto L3 goto FALSE
L3	if $e < f$ goto TRUE goto FALSE
TRUE	Boolean result = true
FALSE	Boolean result = false.